

CHEAT SHEETS

A Complete Reference for Modern Git

- Setup & Config
- Staging & Commits
- Branching
- Remotes
- History & Blame
- Undoing Changes
- Rebase
- Hooks
- Internals
- Workflows

12 REFERENCE SHEETS

CONTENTS

FOUNDATIONS	INTERMEDIATE	ADVANCED
01 Setup & Configuration	05 Inspecting History	09 Workflows & Collaboration
02 Core Workflow	06 Undoing Changes	10 Hooks & Automation
03 Branching	07 Stash & Tags	11 Git Internals
04 Remotes & Syncing	08 Advanced Rebase & Merging	12 Power Commands & Tips

Sheet 1

Setup & Configuration

git config · global/local scope · identity · aliases · .gitignore

Sheet 2

Core Workflow

init · clone · status · add · commit · log · staging area

Sheet 3

Branching

branch · checkout · switch · merge · rebase · HEAD · detached HEAD

Sheet 4

Remotes & Syncing

remote · fetch · pull · push · tracking branches · origin · upstream

Sheet 5

Inspecting History

log · diff · show · blame · bisect · shortlog · grep

Sheet 6

Undoing Changes

restore · reset · revert · clean · amend · reflog · detached HEAD

Sheet 7

Stash & Tags

stash · apply · pop · drop · tag · annotated · lightweight · push tags

Sheet 8

Advanced Rebase & Merging

rebase · interactive rebase · squash · cherry-pick · merge strategies · conflicts

Sheet 9

Workflows & Collaboration

GitHub Flow · Git Flow · trunk-based · PRs · forks · code review · conventions

Sheet 10

Hooks & Automation

pre-commit · commit-msg · post-merge · pre-push · husky · hook scripts

Sheet 11

Git Internals

objects · blobs · trees · commits · refs · packfiles · reflog · gc · plumbing

Sheet 12

Power Commands & Tips

worktree · submodule · sparse-checkout · filter-repo · aliases · bundle

WHAT IS GIT?

Git is a distributed version control system that tracks changes to files over time. Unlike centralized systems, every developer has a complete copy of the repository — including its full history — on their own machine. Git was created by **Linus Torvalds** in 2005 for Linux kernel development and has since become the universal standard for source control. Configuration is stored in plain-text files at three scopes: system (all users), global (current user), and local (current repo), with narrower scopes overriding broader ones.

KEY FACTS

- › `~/.gitconfig` — global config file (user-level)
- › `.git/config` — local config file (repo-level)
- › Local settings always override global settings
- › `git config --list` shows all active settings and their source
- › Run `git --version` to verify your installation
- › Install via `brew install git`, `apt install git`, or `git-scm.com`

FIRST-TIME SETUP & ALIASES

```
# — Minimum required after installing Git —————
git config --global user.name "Ada Lovelace"
git config --global user.email "ada@example.com"

# — Editor: VS Code, Vim, Nano —————
git config --global core.editor "code --wait" # VS Code
git config --global core.editor "vim"        # Vim
git config --global core.editor "nano"       # Nano

# — Useful aliases —————
git config --global alias.st status
git config --global alias.co checkout
git config --global alias.br branch
git config --global alias.last "log -1 HEAD --stat"
git config --global alias.lg "log --oneline --graph --decorate --all"

# — Use aliases like regular commands —————
git st          # same as: git status
git lg          # compact decorated graph log

# — Per-repo override (inside the repo directory) —————
git config --local user.email "work@company.com"

# — Remove a setting —————
git config --global --unset alias.st

# — Global .gitignore (applies to every repo) —————
git config --global core.excludesFile ~/.gitignore_global
echo ".DS_Store" >> ~/.gitignore_global
echo "Thumbs.db" >> ~/.gitignore_global
```

NOTES

- `--global` — writes to `~/.gitconfig`; affects every repo for this user
- `--local` — writes to `.git/config`; overrides global for this repo only
- `alias.lg` — the quoted value is the full git command minus the `git` prefix
- `--wait` — tells VS Code to block the terminal until you close the editor tab
- `core.excludesFile` — a global gitignore applied on top of every repo's local `.gitignore`
- `--unset` — removes a single key; use `--remove-section` to drop an entire section

.GITIGNORE PATTERNS

PATTERN	MATCHES
<code>*.log</code>	Any file ending in <code>.log</code> anywhere in the repo
<code>build/</code>	The entire <code>build</code> directory and its contents
<code>/TODO</code>	Only a <code>TODO</code> file at the repo root
<code>doc/**/*.txt</code>	Any <code>.txt</code> file nested anywhere under <code>doc/</code>
<code>!important.log</code>	Un-ignore: track <code>important.log</code> even if <code>*.log</code> is ignored
<code>.env*</code>	Ignore <code>.env</code> , <code>.env.local</code> , <code>.env.production</code> , etc.
<code>node_modules/</code>	Dependency folder — always ignore, never commit

ESSENTIAL GIT CONFIG COMMANDS

COMMAND	DESCRIPTION
<code>git config --global user.name "Name"</code>	Set your display name for all commits
<code>git config --global user.email "e@x.com"</code>	Set your email address for all commits
<code>git config --global core.editor "code --wait"</code>	Set VS Code as the default commit message editor
<code>git config --global init.defaultBranch main</code>	Use <code>main</code> instead of <code>master</code> for new repos
<code>git config --global core.autocrlf input</code>	Normalize line endings on commit (use <code>true</code> on Windows)
<code>git config --global pull.rebase false</code>	Use merge strategy for <code>git pull</code> (recommended default)
<code>git config --global --edit</code>	Open the global config file directly in your editor
<code>git config --list --show-origin</code>	List all settings with the file they come from

COMMON MISTAKES

- › Skipping `user.name` and `user.email` — commits will show blank or wrong identity, which cannot be changed without rewriting history
- › Adding `node_modules/` or `.env` to `.gitignore` after already committing them — the files must also be removed from tracking with `git rm --cached`
- › Using `--system` instead of `--global` — system scope requires root/admin and affects every user on the machine

TIPS

- › Set `init.defaultBranch main` globally once so every `git init` uses `main` without extra steps
- › Keep a `~/.gitignore_global` for OS/editor noise (`.DS_Store`, `.idea/`, `*.swp`) so individual repos stay clean
- › Use `git config --list --show-origin` to debug which file is supplying a setting — essential when local overrides global unexpectedly

SUMMARY

- › `git config --global` — user-level settings in `~/.gitconfig`
- › `git config --local` — repo-level settings that override global
- › `alias.*` — shorthand commands defined in config
- › `.gitignore` — patterns for files Git should never track
- › `core.excludesFile` — a global gitignore for OS and editor clutter

THE THREE-STAGE MODEL

Every file in a Git repo lives in one of three places. The **working tree** is the directory you edit. The **staging area** (also called the index) is a draft of your next commit — you explicitly add changes here before they become permanent. The **repository** is the permanent history stored in `.git/`. Moving changes left to right (add → commit) is the core loop you will repeat thousands of times. Moving right to left (restore, reset) is how you undo.

FILE STATES

- › **Untracked** — new file Git has never seen
- › **Unmodified** — tracked, matches last commit
- › **Modified** — tracked, changed in working tree
- › **Staged** — change is in the index, ready to commit
- › A file can be both staged and modified at once if edited after `git add`
- › `git status -s` shows a compact two-column state: index state | working tree state

NOTES

- `git status -s` — left column = index, right column = working tree; `??` means untracked
- `git add -p` — "patch" mode lets you split one file's changes into separate commits for cleaner history
- `git commit -am` — only stages already-tracked files; new (untracked) files still need a manual `git add`
- `--amend` — rewrites the last commit; never amend commits that have already been pushed to a shared remote
- `git log --oneline --graph` — best mental model of branch topology; pair with the `lg` alias from Sheet 01
- `git show HEAD` — `HEAD` always refers to the current commit; `HEAD~1` is one before it

A FULL COMMIT CYCLE

```
# — Start a project —————
git init my-app && cd my-app

# — Check state at any time —————
git status                # full output
git status -s             # short: XY filename
                        # ?? = untracked  M = modified  A = staged

# — Stage and commit —————
echo "# My App" > README.md
git add README.md         # stage one file
git commit -m "Initial commit: add README"

# — Make more changes —————
echo "hello" > app.js
git add .                 # stage everything
git status -s             # A  app.js (staged)

# — Amend: forgot a file before pushing —————
echo "*.log" > .gitignore
git add .gitignore
git commit --amend --no-edit # add to last commit, keep message

# — Inspect history —————
git log                   # full log
git log --oneline         # one line per commit
git log --oneline --graph # with branch graph
git log -5               # last 5 commits only
git log -- app.js        # commits that touched app.js

# — Show a single commit —————
git show HEAD             # diff + metadata of latest commit
git show abc1234         # show a specific commit by hash
```

STARTING A REPOSITORY

COMMAND	DESCRIPTION
<code>git init</code>	Create a new repo in the current directory
<code>git init my-project</code>	Create a new repo inside a new <code>my-project/</code> folder
<code>git clone <url></code>	Download a repo and its full history
<code>git clone <url> my-dir</code>	Clone into a custom directory name
<code>git clone --depth 1 <url></code>	Shallow clone — latest snapshot only, no full history

STAGING VARIANTS

COMMAND	DESCRIPTION
<code>git add file.txt</code>	Stage a single file
<code>git add src/</code>	Stage all changes inside a directory
<code>git add .</code>	Stage all changes in the working tree
<code>git add -p</code>	Interactive: stage individual hunks within a file
<code>git restore --staged file</code>	Unstage a file without losing the working tree change
<code>git commit -m "msg"</code>	Commit staged changes with an inline message
<code>git commit -am "msg"</code>	Stage all tracked modifications and commit in one step
<code>git commit --amend</code>	Rewrite the last commit's message or add forgotten files

COMMON MISTAKES

- › Using `git commit -am` expecting it to stage new files — `-a` only auto-stages files already tracked by Git
- › Amending a commit that was already pushed — this rewrites history and forces collaborators to reset their branches
- › Writing vague messages like "fix" or "update" — a good message completes the sentence "If applied, this commit will..."

TIPS

- › Use `git add -p` to review your own diff before staging — it catches debug code and half-finished changes before they land in history
- › Run `git status` before every commit; it takes one second and eliminates most "oops I staged the wrong thing" moments
- › Keep commits atomic — one logical change per commit. It makes `git bisect`, `git revert`, and code review dramatically easier

SUMMARY

- › `git init` / `git clone` — start a repo locally or from remote
- › `git add` — move changes from working tree → staging area
- › `git commit` — move staged changes → permanent history
- › `git status -s` — compact two-column file state view
- › `git log --oneline --graph` — visual history at a glance

BRANCHES & HEAD

A **branch** in Git is simply a lightweight movable pointer to a commit. Creating one costs almost nothing — it is a 41-byte file. **HEAD** is a special pointer that tells Git which branch you are currently on; when you commit, HEAD's branch moves forward automatically. `git switch` (Git 2.23+) is the modern replacement for `git checkout` when changing branches. **Detached HEAD** occurs when HEAD points directly at a commit instead of a branch name — commits made here are not on any branch and can be lost.

KEY FACTS

- › `main` / `master` — default branch name (just a convention)
- › Branches are local until pushed to a remote
- › `HEAD~1` — parent of HEAD; `HEAD~3` — three commits back
- › `HEAD^` — first parent; `HEAD^2` — second parent (merge commits)
- › Fast-forward merge: no new commit, pointer just moves
- › Three-way merge: creates a new merge commit with two parents

NOTES

- `switch -c` — replaces the older `checkout -b`; prefer `switch` for branch operations
- `--no-ff` — "no fast-forward"; always creates a merge commit, making the branch visible in history
- `--squash` — combines all branch commits into one staged change; you still run `git commit` manually
- Conflict markers — `<<<<<<` / `=====` / `>>>>>>` — must be removed before staging
- `--detach` — safe way to inspect old commits without accidentally committing on a real branch
- `--graph --all` — shows every branch in the log, not just the currently checked-out one

BRANCH & MERGE WORKFLOW

```
# — Create and work on a feature branch —————
git switch -c feature/login # create + switch
git add auth.js
git commit -m "Add login form"
git add session.js
git commit -m "Add session handling"

# — View branch topology —————
git log --oneline --graph --all
# * a1b2c3d (feature/login) Add session handling
# * d4e5f6a Add login form
# * 9f8e7d6 (HEAD -> main) Initial commit

# — Merge back into main —————
git switch main
git merge feature/login # fast-forward if no main commits since branch

# — Merge with a commit (preserves branch in history) —————
git merge --no-ff feature/login -m "Merge feature/login"

# — Handle a merge conflict —————
git merge feature/login # CONFLICT in auth.js
# 1. Open auth.js, resolve conflict markers:
# <<<<<< HEAD
# your version
# =====
# their version
# >>>>>> feature/login
git add auth.js # mark resolved
git commit # complete the merge

# — Detached HEAD: check out a commit directly —————
git switch --detach HEAD~3 # look at older state safely
git switch main # return to branch (detached commits lost)
```

BRANCH COMMANDS

COMMAND	DESCRIPTION
<code>git branch</code>	List local branches; current branch is starred
<code>git branch -a</code>	List local and remote-tracking branches
<code>git branch feature</code>	Create a branch at the current commit
<code>git switch feature</code>	Switch to the feature branch (updates working tree)
<code>git switch -c feature</code>	Create and switch in one step
<code>git branch -d feature</code>	Delete a merged branch (safe — refuses if unmerged)
<code>git branch -D feature</code>	Force-delete even if unmerged
<code>git branch -m old new</code>	Rename a branch

MERGE COMMANDS

COMMAND	DESCRIPTION
<code>git merge feature</code>	Merge feature into the current branch
<code>git merge --no-ff feature</code>	Always create a merge commit even if fast-forward is possible
<code>git merge --squash feature</code>	Squash all feature commits into one staged change
<code>git merge --abort</code>	Abort a conflicted merge and restore pre-merge state
<code>git diff --merge-base</code>	Show changes since the branch diverged
<code>git log --merges</code>	Show only merge commits in history

COMMON MISTAKES

- › Committing in detached HEAD state — those commits have no branch reference and will be garbage-collected; always run `git switch -c new-branch` before committing
- › Forgetting to switch back to `main` before merging — you merge into whatever branch you are currently on
- › Using `git branch -D` without verifying the branch is merged — run `git log main..feature` first to confirm no commits would be lost

TIPS

- › Use descriptive branch names with a prefix: `feature/`, `fix/`, `chore/` — they sort nicely and convey intent at a glance
- › Keep feature branches short-lived; the longer a branch diverges, the harder the eventual merge becomes
- › Run `git log --oneline --graph --all` (or your `lg` alias) after every merge to confirm the topology looks correct

SUMMARY

- › `git switch -c` — create and switch to a new branch
- › `git merge` — integrate a branch into the current branch
- › `--no-ff` — preserve branch structure in history
- › `HEAD` — pointer to the current branch (or commit in detached mode)
- › Resolve conflicts by editing files, then `git add + git commit`

REMOTES & TRACKING BRANCHES

A **remote** is a named reference to another copy of the repository, typically on a server.

The default remote created by `git clone` is called **origin**. Git stores a local read-only snapshot of each remote branch called a **remote-tracking branch** (e.g. `origin/main`). These snapshots are updated by `git fetch` but never touched by your local commits. `git pull` is shorthand for `git fetch` followed by `git merge` (or `git rebase` if configured). Understanding the difference between fetch and pull is essential for safe collaboration.

KEY FACTS

- › `origin` — conventional name for the primary remote
- › `upstream` — conventional name for the original repo when working with forks
- › `origin/main` — remote-tracking branch; read-only local snapshot
- › `git fetch` is always safe — it never changes your working tree
- › A tracking relationship links a local branch to a remote branch for push/pull defaults
- › HTTPS vs SSH: SSH requires key setup but avoids password prompts

NOTES

- `-u` / `--set-upstream` — only needed once per branch; after that `git push` and `git pull` work with no arguments
- `HEAD..origin/main` — the double-dot range means "commits reachable from `origin/main` but not from `HEAD`"
- `--force-with-lease` — always prefer this over `--force`; it won't overwrite teammates' commits you haven't fetched
- `--prune` — removes `origin/feature` remote-tracking branches that no longer exist on the server
- `upstream` remote — a second remote pointing at the repo you forked from, keeping your fork in sync with the original
- `git pull --rebase` replays your local commits on top of fetched changes, avoiding a spurious merge commit

TYPICAL REMOTE WORKFLOWS

```
# — Connect a local repo to GitHub —————
git remote add origin git@github.com:user/repo.git
git push -u origin main      # -u sets tracking; after this just: git push

# — Fetch then inspect before merging —————
git fetch origin             # update origin/* snapshots
git log HEAD..origin/main    # commits on remote that you don't have
git diff HEAD origin/main    # diff your work vs remote
git merge origin/main        # merge when ready

# — Pull with rebase (preferred for cleaner history) —————
git pull --rebase origin main
# equivalent to:
git fetch origin
git rebase origin/main

# — Work with a fork (open-source contribution) —————
git remote add upstream git@github.com:original/repo.git
git fetch upstream           # get upstream changes
git merge upstream/main      # sync your fork's main

# — Push a feature branch and delete after merge —————
git push -u origin feature/login
# ... PR reviewed and merged on GitHub ...
git fetch --prune            # remove stale remote-tracking branches
git branch -d feature/login  # delete local branch

# — Recover from rejected push —————
git pull --rebase            # rebase your commits on top of remote
git push                     # now succeeds
```

COMMON MISTAKES

- › Using `git push --force` on a shared branch — this rewrites history for everyone; always use `--force-with-lease` and only on branches you alone own
- › Confusing `git fetch` and `git pull` — fetch is always safe; pull immediately integrates changes and can create conflicts
- › Forgetting `-u` on the first push — the next `git pull` will fail with "no tracking information" until a tracking relationship exists

TIPS

- › Prefer `git fetch` + inspect + `git merge` over bare `git pull` — you see exactly what's coming before integrating it
- › Set `git config --global pull.rebase true` to make `git pull` always rebase instead of merge, keeping history linear
- › Run `git fetch --prune` regularly on long-lived projects; stale `origin/*` branches accumulate and clutter branch listings

REMOTE MANAGEMENT

COMMAND	DESCRIPTION
<code>git remote -v</code>	List remotes with their fetch and push URLs
<code>git remote add origin <url></code>	Add a remote named origin
<code>git remote rename origin upstream</code>	Rename a remote
<code>git remote remove origin</code>	Remove a remote reference
<code>git remote set-url origin <url></code>	Change a remote's URL (e.g. HTTPS → SSH)
<code>git remote show origin</code>	Inspect tracking relationships and remote branch state

FETCH · PULL · PUSH

COMMAND	DESCRIPTION
<code>git fetch</code>	Download all remote changes without merging
<code>git fetch origin main</code>	Fetch only the remote main branch
<code>git pull</code>	Fetch + merge the tracking branch into current
<code>git pull --rebase</code>	Fetch + rebase instead of merge (cleaner history)
<code>git push origin main</code>	Push local main to remote origin
<code>git push -u origin feature</code>	Push and set tracking relationship (<code>-u</code> = <code>--set-upstream</code>)
<code>git push --force-with-lease</code>	Safer force push — fails if remote has new commits you haven't seen
<code>git push origin --delete feature</code>	Delete a remote branch

SUMMARY

- › `git remote add` — connect to a remote repository
- › `git fetch` — download changes without modifying working tree
- › `git pull --rebase` — fetch + rebase for clean linear history
- › `git push -u` — push and set tracking branch in one step
- › `--force-with-lease` — safe alternative to `--force`

READING THE REPOSITORY'S PAST

Git stores every snapshot forever, making history fully inspectable. `git log` walks the commit graph; `git diff` compares any two states; `git blame` annotates every line of a file with the commit that last changed it; `git bisect` performs a binary search across commits to find the exact commit that introduced a bug. All of these are read-only — nothing inspected here changes the repository. Combine `--oneline`, `--graph`, and `--all` flags to get a compact visual map of any project's history.

REVISION SYNTAX

- › `HEAD` — current commit
- › `HEAD~3` — 3 commits before HEAD
- › `abc1234` — commit by partial hash (min 4 chars)
- › `main@{2}` — where `main` pointed 2 moves ago
- › `HEAD..feature` — commits in feature not in HEAD
- › `HEAD...feature` — commits in either, not both (symmetric difference)

NOTES

- `git blame -L` — line range limits annotation to the most relevant section; combine with `-w` to skip pure whitespace commits
- `git bisect run` — fully automated; test script must exit 0 for "good" and any non-zero for "bad"; bisects $\log_2(N)$ commits automatically
- `-S "string"` — the "pickaxe" flag; finds commits where the string count changed (i.e. was added or removed)
- `-G "regex"` — finds commits where the diff text matches the regex, including line rearrangements
- `git shortlog -sn` — the `-s` suppresses commit messages, `-n` sorts by count descending
- `--diff-filter=D` — filter log to only commits that deleted files; also: A (added), M (modified), R (renamed)

BLAME, BISECT & SHORTLOG

```
# — Annotate who changed each line —————
git blame auth.js           # full annotation
git blame -L 40,60 auth.js  # only lines 40-60
git blame -w auth.js        # ignore whitespace-only changes

# — Find which commit introduced a bug —————
git bisect start
git bisect bad              # current commit has the bug
git bisect good v1.2.0      # this tag was known-good
# Git checks out a commit halfway between — test it, then:
git bisect bad              # still broken
git bisect good             # fixed here
# Git narrows in ~log₂(N) steps and prints the culprit commit
git bisect reset           # return to HEAD when done

# — Automate bisect with a test script —————
git bisect run npm test     # exits 0 = good, non-zero = bad

# — Contribution summary —————
git shortlog -sn            # commit count per author, sorted
git shortlog -sn --since="2024-01-01"

# — Search commit content (not just messages) —————
git log -S "secretToken"    # commits that added/removed string
git log -G "regex.*pattern" # commits where diff matches regex
git grep "TODO" HEAD        # grep working tree at HEAD
```

GIT LOG FLAGS

FLAG	EFFECT
<code>--oneline</code>	One line per commit: hash + message
<code>--graph</code>	ASCII branch/merge topology diagram
<code>--all</code>	Include all branches, not just current
<code>--stat</code>	Show files changed and insertion/deletion counts
<code>-p</code>	Show full diff patch for every commit
<code>-n 10</code>	Limit to the last 10 commits
<code>--author="Ada"</code>	Filter by author name or email (partial match)
<code>--since="2 weeks ago"</code>	Filter by date (also: <code>--until</code> , <code>--after</code> , <code>--before</code>)
<code>--grep="login"</code>	Filter commits whose message contains the string
<code>-- path/file</code>	Limit to commits that touched a specific file or path

GIT DIFF VARIANTS

COMMAND	COMPARES
<code>git diff</code>	Working tree vs staging area (unstaged changes)
<code>git diff --staged</code>	Staging area vs last commit (what will be committed)
<code>git diff HEAD</code>	Working tree vs last commit (all uncommitted changes)
<code>git diff main feature</code>	Two branches tip-to-tip
<code>git diff abc1234 def5678</code>	Two specific commits
<code>git diff HEAD~3 HEAD -- file</code>	One file across a range of commits
<code>git diff --stat</code>	Summary of changed files without full patch

COMMON MISTAKES

- › Running `git diff` expecting to see staged changes — use `git diff --staged` for that; bare `git diff` shows only unstaged changes
- › Forgetting `git bisect reset` after a session — leaves you in detached HEAD state on the culprit commit
- › Using `git log --grep` to search code content — `-grep` searches commit messages only; use `-S` or `-G` to search diffs

TIPS

- › Combine flags: `git log --oneline --graph --all --decorate` gives the most useful single-screen history view — save it as an alias
- › Use `git log -- path/to/file` to track a file's history even across renames by adding `-f` or `-follow`
- › `git bisect run` with a test script is dramatically faster than manual bisect for large repos — write the test first

SUMMARY

- › `git log --oneline --graph --all` — compact visual history
- › `git diff --staged` — what will be in the next commit
- › `git blame -L` — per-line authorship for a file range
- › `git bisect` — binary search to find a bug-introducing commit
- › `-S "string"` — find commits that added or removed a string

CHOOSING THE RIGHT UNDO

Git offers several undo mechanisms that operate at different levels of permanence. `git restore` discards working tree or staged changes without touching history. `git reset` moves the branch pointer backward, optionally changing the index and working tree. `git revert` creates a new commit that undoes a previous one — the only safe undo for shared history because it does not rewrite commits.

The golden rule: never rewrite history that has already been pushed to a shared remote branch.

Use `git reflog` as your safety net — it remembers every position HEAD has been at for 90 days.

RESET MODES

- › `--soft` — moves branch pointer; keeps index and working tree unchanged
- › `--mixed` — (default) moves pointer, resets index; keeps working tree
- › `--hard` — moves pointer, resets index and working tree; **destructive**
- › After `--hard`: recover via `git reflog` within 90 days
- › `git revert` is always safe on shared branches
- › `git clean -n` dry-run — shows what would be deleted

NOTES

- `reset --soft` — useful to re-commit with a better message; your changes stay staged and ready
- `reset --mixed` — default; collapses commits back to working tree changes you can re-stage selectively
- `reset --hard` — truly destructive for uncommitted work; always check `git status` first
- `git revert --no-commit` — stages all reverts without committing; lets you combine multiple reverts into one commit
- `git reflog` — records every time HEAD moved; the ultimate recovery tool even after `--hard` resets
- `HEAD@{1}` — reflog syntax meaning "where HEAD was 1 move ago"; safe to branch from or reset to

RESET · REVERT · REFLOG

```
# — Undo last commit, keep changes staged —
git reset --soft HEAD~1
# branch moves back; index unchanged

# — Undo last commit, unstage changes (most common) —
git reset HEAD~1
# --mixed is default
git reset HEAD~1 -- file.txt
# reset only one file in index

# — Nuke last commit and all changes (dangerous) —
git reset --hard HEAD~1
# recover if needed:
git reflog
# find the orphaned commit hash, e.g. abc1234
git reset --hard abc1234
# restore to it

# — Safe undo for shared branches —
git revert HEAD
# new commit that undoes HEAD
git revert abc1234
# undo a specific older commit
git revert HEAD~3..HEAD
# undo a range of commits (3 new reverts)
git revert --no-commit HEAD~3..HEAD
# stage reverts, commit manually

# — Use reflog as a safety net —
git reflog
# every HEAD movement for 90 days
# HEAD@{0}: reset --hard HEAD~1
# HEAD@{1}: commit: Add login form ← the commit we lost
git switch -c recovery HEAD@{1}
# recover on a new branch
```

UNDO COMMANDS AT A GLANCE

COMMAND	TOUCHES HISTORY?	SAFE TO SHARE?
<code>git restore file</code>	No	✓
<code>git restore --staged file</code>	No	✓
<code>git commit --amend</code>	Yes — rewrites	✗ if pushed
<code>git reset --soft HEAD~1</code>	Yes — rewrites	✗ if pushed
<code>git reset --hard HEAD~1</code>	Yes — rewrites	✗ if pushed
<code>git revert HEAD</code>	Yes — adds commit	✓
<code>git clean -fd</code>	No	✓

RESTORE & CLEAN VARIANTS

COMMAND	DESCRIPTION
<code>git restore file.txt</code>	Discard working tree changes to file (irreversible)
<code>git restore .</code>	Discard all working tree changes
<code>git restore --staged file</code>	Unstage a file (keeps working tree change)
<code>git restore --source HEAD~2 file</code>	Restore a file to its state 2 commits ago
<code>git clean -n</code>	Dry run: show untracked files that would be removed
<code>git clean -fd</code>	Remove untracked files (<code>-f</code>) and directories (<code>-d</code>)
<code>git clean -fdx</code>	Also remove ignored files (full clean build state)

COMMON MISTAKES

- › Using `git reset --hard` on a shared branch — this rewrites public history and forces every collaborator to manually reset their local branch
- › Running `git clean -fd` without a `-n` dry run first — untracked files are deleted permanently with no git recovery path
- › Using `git restore file` when you meant to unstage — `restore` without `--staged` discards working tree changes irreversibly

TIPS

- › When in doubt on a shared branch, reach for `git revert` — it creates a paper trail and never forces teammates to change their history
- › Always run `git reflog` before panicking after a hard reset — the "lost" commits are almost certainly still there for 90 days
- › Use `git reset --soft HEAD~1` to break a large commit into smaller ones: uncommit, re-add with `-p`, recommit in logical chunks

SUMMARY

- › `git restore` — discard working tree / staged changes safely
- › `git reset --soft/--mixed/--hard` — move branch pointer back
- › `git revert` — safe undo for shared branches; adds a new commit
- › `git clean -fd` — remove untracked files and directories
- › `git reflog` — 90-day safety net for all HEAD movements

STASH & TAGS

Stash is a temporary shelf for uncommitted changes. It saves your working tree and index to a stack so you can switch context cleanly, then restore the changes later. Each stash entry is identified as `stash@{0}` (most recent), `stash@{1}`, etc. **Tags** mark specific commits permanently — typically release points. A **lightweight tag** is just a named pointer to a commit. An **annotated tag** is a full Git object with a tagger, date, message, and optional GPG signature; prefer annotated for releases because they carry metadata and can be signed.

KEY FACTS

- › Stash is a stack — `stash@{0}` is always the most recent entry
- › `git stash pop` = apply + drop; `git stash apply` keeps the stash entry
- › Untracked and ignored files are *not* stashed by default
- › Tags are not pushed with `git push` — must use `git push --tags` or `git push origin v1.0`
- › Annotated tags store tagger, timestamp, and message
- › Use `git describe` to get the nearest tag relative to a commit

NOTES

- `stash push -- files` — partial stash of specific files only; the rest remain in the working tree
- `stash branch` — the safest way to apply a stash that may conflict with current work; starts fresh on a new branch
- `-u` flag — includes untracked files; always use when switching branches with new files not yet added
- `-a` annotated tag — stores a full object in Git's database; can be GPG-signed with `-s` instead of `-a`
- `git describe` output — format is `tag-N-gHASH`: tag name, commits since tag, abbreviated hash
- Tags are permanent references — deleting a pushed tag should be coordinated with teammates who may have fetched it

STASH & TAG IN PRACTICE

```
# — Interrupt work to fix a hotfix —
git stash push -m "WIP: login redesign"
git switch main
git switch -c fix/crash
# ... make fix ...
git commit -m "Fix null pointer in session"
git switch main
git merge fix/crash

# — Return to your feature —
git switch feature/login
git stash list
# stash@{0}: On feature/login: WIP: login redesign
git stash pop           # restores and removes entry

# — Stash only specific files —
git stash push -m "auth only" -- auth.js session.js

# — Apply a stash to a different branch —
git stash branch experiment stash@{0}
# creates branch, checks it out, applies and drops the stash

# — Tag a release —
git switch main
git tag -a v2.1.0 -m "Release 2.1.0: OAuth support"
git push origin v2.1.0

# — Tag an older commit —
git log --oneline
# abc1234 Merge feature/oauth
git tag -a v2.0.0 abc1234 -m "Release 2.0.0"
git push origin v2.0.0

# — Describe current state relative to a tag —
git describe --tags
# v2.1.0-4-gabc1234 → 4 commits after v2.1.0, at abc1234
```

STASH COMMANDS

COMMAND	DESCRIPTION
<code>git stash</code>	Stash working tree + index changes
<code>git stash push -m "msg"</code>	Stash with a descriptive message
<code>git stash -u</code>	Also stash untracked files
<code>git stash -a</code>	Stash everything including ignored files
<code>git stash list</code>	Show all stash entries
<code>git stash show stash@{1}</code>	Show summary of a specific stash entry
<code>git stash apply</code>	Restore most recent stash, keep entry on stack
<code>git stash pop</code>	Restore and remove most recent stash entry
<code>git stash drop stash@{1}</code>	Delete a specific stash entry
<code>git stash clear</code>	Delete all stash entries
<code>git stash branch fix/hotfix</code>	Create a branch from stash and apply it there

TAG COMMANDS

COMMAND	DESCRIPTION
<code>git tag</code>	List all local tags
<code>git tag v1.0.0</code>	Create a lightweight tag at HEAD
<code>git tag -a v1.0.0 -m "msg"</code>	Create an annotated tag with a message
<code>git tag -a v1.0.0 abc1234</code>	Tag a specific past commit
<code>git show v1.0.0</code>	Show tag metadata and the tagged commit
<code>git push origin v1.0.0</code>	Push a single tag to remote
<code>git push origin --tags</code>	Push all local tags to remote
<code>git tag -d v1.0.0</code>	Delete a local tag
<code>git push origin --delete v1.0.0</code>	Delete a remote tag
<code>git describe --tags</code>	Describe HEAD relative to nearest tag

COMMON MISTAKES

- › Using `git stash` without `-u` when you have new untracked files — those files are not stashed and may cause conflicts when you pop later
- › Forgetting to push tags after a release — tags exist only locally until explicitly pushed; teammates and CI systems won't see them
- › Accumulating stash entries without names — after a week `stash@{3}` is meaningless; always use `git stash push -m "description"`

TIPS

- › Treat stash as a short-term buffer, not long-term storage — work-in-progress branches (`git switch -c wip/feature`) are safer for multi-day interruptions
- › Use annotated tags (`-a`) for every release, not lightweight tags — they carry authorship and a message, and are required for tools like `git describe` to work well
- › Use `git stash show -p stash@{0}` to review a stash as a full diff before applying it

SUMMARY

- › `git stash push -m` — shelve work-in-progress with a label
- › `git stash pop` — restore and remove the top stash entry
- › `-u` — include untracked files in the stash
- › `git tag -a` — annotated tag with message and metadata
- › `git push origin --tags` — tags must be pushed separately

REBASE VS MERGE

`git rebase` replays your commits on top of another branch, producing a **linear history** as if development happened sequentially. `git merge` preserves the true branching topology with a merge commit. Both integrate changes; the difference is cosmetic and philosophical. **Interactive rebase** (`-i`) is a powerful history editor: squash noisy commits, reword messages, reorder or drop commits entirely — all before sharing. `git cherry-pick` copies one or more commits from any branch onto the current branch without a full merge. *Never rebase shared/public branches.*

INTERACTIVE REBASE ACTIONS

- › `pick` — keep commit as-is
- › `reword` — keep commit, edit message
- › `edit` — pause to amend the commit
- › `squash` — combine with previous commit, merge messages
- › `fixup` — like squash, but discard this commit's message
- › `drop` — remove the commit entirely from history

NOTES

- `fixup` vs `squash` — both combine commits; `fixup` silently discards the squashed message, `squash` opens the editor to merge messages
- `rebase --continue` — after resolving each conflict file, `git add` it, then continue; do not `git commit` mid-rebase
- `rebase --onto` — three-argument form: new base, old base (exclusive), branch tip; powerful for re-parenting work
- `cherry-pick -n` — useful when you want to cherry-pick several commits and combine them into one before committing
- `-X theirs/ours` — strategy option, not strategy; resolves conflicts automatically but still creates a merge commit
- After rebasing, force-push the branch with `--force-with-lease` — hashes change so the remote ref must be updated

INTERACTIVE REBASE & CHERRY-PICK

```
# — Clean up feature branch before opening a PR —————
git switch feature/login
git rebase -i HEAD~5
# Editor opens with last 5 commits:
# pick a1b2c3d Add login form
# pick d4e5f6a WIP: fix typo
# pick 789abcd WIP: another fix
# pick bcdef01 Add session handling
# pick 1234567 Add tests
#
# Change to:
# pick a1b2c3d Add login form
# fixup d4e5f6a WIP: fix typo      ← squash into previous, drop msg
# fixup 789abcd WIP: another fix   ← squash into previous, drop msg
# reword bcdef01 Add session handling ← keep but edit the message
# pick 1234567 Add tests
# Save and close → Git replays, pauses on "reword" for new message

# — Rebase onto updated main —————
git fetch origin
git rebase origin/main          # replay your commits on remote main
# Resolve any conflicts, then:
git add conflicted-file.txt
git rebase --continue

# — Cherry-pick a hotfix commit onto a release branch —————
git switch release/2.0
git cherry-pick abc1234         # copy one commit
git cherry-pick abc1234 def5678 # copy multiple commits

# — --onto: move a sub-branch to a different base —————
# Situation: feature/ui was branched from feature/api (not main)
# Move feature/ui commits so they start from main instead:
git rebase --onto main feature/api feature/ui
```

REBASE COMMANDS

COMMAND	DESCRIPTION
<code>git rebase main</code>	Replay current branch commits on top of main
<code>git rebase -i HEAD~4</code>	Interactively edit the last 4 commits
<code>git rebase -i main</code>	Interactively edit all commits since branching from main
<code>git rebase --continue</code>	Continue after resolving a conflict mid-rebase
<code>git rebase --abort</code>	Cancel and return to the pre-rebase state
<code>git rebase --skip</code>	Skip a conflicting commit and continue
<code>git rebase --onto new old branch</code>	Re-parent a branch range onto a different base

CHERRY-PICK & MERGE STRATEGIES

COMMAND	DESCRIPTION
<code>git cherry-pick abc1234</code>	Apply commit abc1234 onto the current branch
<code>git cherry-pick abc..def</code>	Apply a range of commits (exclusive of abc)
<code>git cherry-pick -n abc1234</code>	Stage the change without committing (<code>--no-commit</code>)
<code>git cherry-pick --abort</code>	Cancel a conflicted cherry-pick
<code>git merge -s ours feature</code>	Merge strategy: keep current branch content entirely
<code>git merge -X theirs feature</code>	Merge strategy option: auto-resolve conflicts with their side
<code>git merge -X ours feature</code>	Merge strategy option: auto-resolve conflicts with our side

COMMON MISTAKES

- › Rebasing `main` or any shared branch — rebase rewrites commit hashes; anyone who based work on the old hashes must reconcile the divergence manually
- › Running `git commit` during a rebase conflict — instead, resolve the conflict, `git add`, then `git rebase --continue`
- › Dropping a commit in interactive rebase by accident — recover via `git reflog` to find the original hash before the rebase

TIPS

- › Make a backup branch before any interactive rebase: `git branch backup/feature` — you can always `git reset --hard backup/feature` if it goes wrong
- › Use `git rebase -i main` (not `HEAD~N`) so the rebase scope automatically covers exactly the commits since you branched
- › Cherry-pick is ideal for backporting fixes to release branches; keep a note of which commits are cherry-picked so the team doesn't apply them twice

SUMMARY

- › `git rebase main` — replay commits for linear history
- › `git rebase -i` — edit, squash, reorder, or drop commits
- › `fixup` — squash into previous commit, discard message
- › `git cherry-pick` — copy specific commits onto current branch
- › Never rebase branches shared with others

CHOOSING A WORKFLOW

A **Git workflow** is a convention for how branches are named, when merges happen, and who can push where. The right workflow depends on team size, release cadence, and deployment model. **GitHub Flow** is simple: branch from `main`, open a pull request, merge after review. **Git Flow** adds structured long-lived branches for releases and hotfixes — useful for scheduled releases. **Trunk-based development** has everyone commit to `main` frequently behind feature flags — common at high-velocity teams. Whichever you pick, consistency matters more than the specific model.

GIT FLOW BRANCH TYPES

- › `main` — always production-ready; only receives merges from release/hotfix
- › `develop` — integration branch; features merge here first
- › `feature/*` — branches from `develop`, merges back to `develop`
- › `release/*` — branches from `develop`; only bugfixes; merges to `main` + `develop`
- › `hotfix/*` — branches from `main`; emergency fix; merges to `main` + `develop`

WORKFLOW COMPARISON

WORKFLOW	LONG-LIVED BRANCHES	BEST FOR
GitHub Flow	main only	Continuous deployment, small teams
Git Flow	main + develop	Scheduled releases, versioned software
Trunk-based	main only	Large teams, feature flags, CI/CD
Forking	main (per fork)	Open-source, external contributors

COMMIT MESSAGE CONVENTIONS

TYPE	MEANING (CONVENTIONAL COMMITS)
feat	A new feature
fix	A bug fix
docs	Documentation changes only
style	Formatting, missing semicolons — no logic change
refactor	Code restructuring without feature or fix
test	Adding or updating tests
chore	Build process, tooling, dependency updates
BREAKING CHANGE	Footer note for incompatible API changes

GITHUB FLOW STEP-BY-STEP

STEP	ACTION
1. Branch	<code>git switch -c feature/name</code> from up-to-date <code>main</code>
2. Commit	Small, frequent commits with clear messages
3. Push	<code>git push -u origin feature/name</code>
4. PR	Open a pull request; describe what and why
5. Review	Address feedback with additional commits on the branch
6. Merge	Squash or merge into <code>main</code> after approval
7. Deploy	CI/CD deploys <code>main</code> ; delete the feature branch

FORK & PR WORKFLOW

STEP	ACTION
1. Fork	Fork on GitHub; clone your fork locally
2. Upstream	<code>git remote add upstream <original-url></code>
3. Sync	<code>git fetch upstream</code> && <code>git merge upstream/main</code>
4. Branch	Work on a feature branch in your fork
5. Push	Push to your fork: <code>git push origin feature</code>
6. PR	Open PR from fork's branch to original repo's <code>main</code>
7. Sync	Keep in sync during review: fetch upstream, rebase

COMMON MISTAKES

- › Opening a PR from `main` in your fork — always work on a named feature branch so the PR can be updated without polluting your fork's `main`
- › Mixing workflow models mid-project — pick one and document it in `CONTRIBUTING.md`; mixed conventions create confusion about where to branch from
- › Writing vague PR titles ("various fixes") — the title becomes part of the merge commit message and the permanent changelog; be specific

TIPS

- › Use **Conventional Commits** (`feat:`, `fix:`, `chore:`) — they enable automated changelogs and semantic versioning tools like `semantic-release`
- › Add a `CODEOWNERS` file to auto-request review from the right people based on which files changed in a PR
- › Keep PRs small and focused — a PR touching fewer than 400 lines of changed code gets reviewed faster and more thoroughly than large ones

SUMMARY

- › GitHub Flow — branch, PR, merge to `main`; ideal for CD
- › Git Flow — `main` + `develop` with release branches
- › Fork workflow — fork → upstream remote → PR to original
- › Conventional Commits — typed prefixes enable automation
- › Small, frequent PRs — faster reviews, less merge pain

GIT HOOKS

Git hooks are executable scripts in `.git/hooks/` that Git runs automatically at specific points in the workflow — before or after commits, merges, pushes, and more. A hook is triggered by naming the script after the event (e.g. `pre-commit`) and making it executable. If a **client-side hook exits with a non-zero status**, the Git operation is aborted. Hooks are not committed or cloned with the repository by default, so teams use tools like **Husky** (Node.js) or **pre-commit** (Python) to share and manage hooks through version control.

KEY FACTS

- Hooks live in `.git/hooks/` — not committed to the repo by default
- Must be executable: `chmod +x .git/hooks/pre-commit`
- Exit 0 = success (operation proceeds); non-zero = abort
- Client-side hooks run locally; server-side hooks run on the remote
- `git commit --no-verify` skips all client-side hooks
- Use `core.hooksPath` to point Git to a shared hooks directory

NOTES

- `#!/bin/sh` — shebang line is required; hooks are shell scripts by convention but can be any executable
- `$1` in `commit-msg` — path to the temporary file containing the commit message written by the user
- `diff-tree` in `post-merge` — compares `ORIG_HEAD` (pre-merge) to `HEAD` (post-merge) to detect lockfile changes
- `core.hooksPath` — redirects Git to a committed directory for hooks; each developer runs `git config core.hooksPath scripts/hooks` once after cloning
- Husky automates `core.hooksPath` setup via the `prepare` npm script — runs automatically on `npm install`
- `--no-verify` — bypass hooks when a hook itself is broken; fix the hook and remove this flag before pushing

WRITING & SHARING HOOKS

```
# — Minimal pre-commit hook —
cat > .git/hooks/pre-commit << 'EOF'
#!/bin/sh
npm run lint      # exit 1 if lint fails → commit aborted
npm run test:unit # run fast unit tests only
EOF
chmod +x .git/hooks/pre-commit

# — commit-msg: enforce Conventional Commits format —
cat > .git/hooks/commit-msg << 'EOF'
#!/bin/sh
MSG=$(cat "$1")
PATTERN="^(feat|fix|docs|style|refactor|test|chore)((.+\.))?: .{1,72}"
if ! echo "$MSG" | grep -qE "$PATTERN"; then
  echo "ERROR: Commit message must follow Conventional Commits."
  echo "  Example: feat(auth): add OAuth login"
  exit 1
fi
EOF
chmod +x .git/hooks/commit-msg

# — post-merge: auto-install deps if lockfile changed —
cat > .git/hooks/post-merge << 'EOF'
#!/bin/sh
if git diff-tree -r --name-only --no-commit-id ORIG_HEAD HEAD \
  | grep -q "package-lock.json"; then
  echo "Lockfile changed — running npm install"
  npm install
fi
EOF
chmod +x .git/hooks/post-merge

# — Share hooks via core.hooksPath —
mkdir -p scripts/hooks
cp .git/hooks/pre-commit scripts/hooks/
git config core.hooksPath scripts/hooks # team members set this once
```

COMMON MISTAKES

- Forgetting `chmod +x` — Git silently skips hooks that are not executable, giving no warning that the hook isn't running
- Putting slow tests in `pre-commit` — hooks blocking commits for minutes train developers to use `--no-verify`; save slow tests for `pre-push` or CI
- Relying solely on client-side hooks for security — any developer can bypass them with `--no-verify`; enforce critical policies with server-side hooks or branch protection rules

TIPS

- Use `lint-staged` in `pre-commit` to lint only staged files — dramatically faster than linting the whole project on every commit
- Document hook setup in `README.md` or `CONTRIBUTING.md` — new contributors must know to run `git config core.hooksPath scripts/hooks` after cloning
- Keep hooks fast and focused — one hook per concern is easier to debug and bypass surgically when needed

SUMMARY

- Hooks live in `.git/hooks/` — must be executable scripts
- `pre-commit` — lint/format before commit is written
- `commit-msg` — validate message format after writing
- `core.hooksPath` — share hooks via a committed directory
- Husky — manages hook installation via npm lifecycle

COMMON HOOKS & WHEN THEY FIRE

HOOK	FIRES	COMMON USE
<code>pre-commit</code>	Before commit message is written	Lint, format, run fast tests
<code>prepare-commit-msg</code>	Before editor opens for message	Auto-insert branch name or ticket ID
<code>commit-msg</code>	After message is written	Validate Conventional Commits format
<code>post-commit</code>	After commit completes	Notifications, local logging
<code>pre-push</code>	Before push to remote	Run full test suite, block on failure
<code>post-merge</code>	After a successful merge	Install dependencies if lockfile changed
<code>pre-rebase</code>	Before rebase begins	Warn against rebasing shared branches
<code>pre-receive</code>	Server: before refs are updated	Enforce policies (server-side)
<code>post-receive</code>	Server: after refs are updated	Trigger CI, send notifications (server-side)

HUSKY SETUP (NODE.JS PROJECTS)

COMMAND / FILE	DESCRIPTION
<code>npm install --save-dev husky</code>	Install Husky as a dev dependency
<code>npx husky init</code>	Create <code>.husky/</code> dir and add prepare script
<code>.husky/pre-commit</code>	Shell script that runs on every commit
<code>.husky/commit-msg</code>	Shell script to validate commit messages
<code>npx commitlint --edit \$1</code>	Typical <code>commit-msg</code> hook body using commitlint
<code>npx lint-staged</code>	Typical <code>pre-commit</code> hook — lint only staged files
<code>git commit --no-verify</code>	Bypass all hooks (emergency escape hatch)

THE OBJECT DATABASE

Git's storage model is a **content-addressed object database** in `.git/objects/`. Everything is stored as one of four object types, identified by a SHA-1 hash of their contents. A **blob** stores raw file contents. A **tree** stores a directory listing (names, modes, and blob/tree hashes). A **commit** stores a tree hash, parent commit hashes, metadata, and a message. A **tag object** stores a pointer to any object plus tagger info. **Refs** (`.git/refs/`) are human-readable names for hashes — branches, tags, and HEAD are all refs.

OBJECT TYPES

- › **blob** — file contents (no filename, no metadata)
- › **tree** — directory: list of (mode, name, hash) entries
- › **commit** — tree + parent(s) + author + committer + message
- › **tag** — annotated tag object pointing to any object
- › Identical files across commits share one blob — no duplication
- › Packfiles compress and delta-encode objects for efficiency

NOTES

- `cat-file -p HEAD` — "porcelain" pretty-print; the commit object contains the tree hash, not file contents directly
- `ls-tree` modes — `100644` = regular file, `100755` = executable, `040000` = directory (tree), `120000` = symlink
- Content-addressing — two identical files in different locations store only one blob; Git detects this automatically
- `git fsck --lost-found` — writes dangling blobs/commits to `.git/lost-found/`; useful after an accidental hard reset
- `git gc` — runs automatically by Git periodically; safe to run manually; packs loose objects into packfiles
- Packfiles (`.git/objects/pack/`) store delta-compressed chains of objects — a repository that is 1 GB of files may have a 50 MB packfile

EXPLORING THE OBJECT STORE

```
# — Inspect the latest commit object —————
git cat-file -p HEAD
# tree 4b825dc642cb6eb9a060e54bf8d69288fbee4904
# parent 9f8e7d6c5b4a3f2e1d0c9b8a7f6e5d4c3b2a1f0
# author Ada Lovelace 1704067200 +0000
# committer Ada Lovelace 1704067200 +0000
# Add OAuth login

# — Inspect the tree that commit points to —————
git ls-tree HEAD
# 100644 blob a8c3f1d README.md
# 100644 blob e7b2f4c auth.js
# 040000 tree d9a1b3e src/

# — Read a blob directly —————
git cat-file -p a8c3f1d # prints contents of README.md at HEAD

# — Resolve expressions to hashes —————
git rev-parse HEAD      # → full 40-char hash
git rev-parse HEAD~2    # → hash 2 commits ago
git rev-parse main      # → hash branch main points to
git rev-parse v1.0.0    # → hash of tag (lightweight: commit hash)

# — Find dangling commits (orphans after reset) —————
git fsck --lost-found   # writes dangling objects to .git/lost-found/
git cat-file -p dangling_hash # inspect to decide if worth recovering

# — Maintenance —————
git gc --aggressive    # repack and compress (takes longer)
git gc --prune=now     # also remove all loose objects immediately
```

PLUMBING COMMANDS

COMMAND	DESCRIPTION
<code>git cat-file -t abc1234</code>	Print the object type (blob, tree, commit, tag)
<code>git cat-file -p abc1234</code>	Pretty-print the object contents
<code>git ls-tree HEAD</code>	List the tree object at HEAD (directory snapshot)
<code>git ls-tree -r HEAD</code>	Recursively list all blobs in the current commit
<code>git hash-object file.txt</code>	Compute and print the SHA-1 hash of a file
<code>git rev-parse HEAD</code>	Resolve any ref or expression to a full 40-char hash
<code>git rev-parse --short HEAD</code>	Short (7-char) hash of HEAD
<code>git count-objects -v</code>	Count loose objects and packfile sizes

REFS & MAINTENANCE

COMMAND / PATH	DESCRIPTION
<code>.git/HEAD</code>	Points to current branch ref (or a commit in detached mode)
<code>.git/refs/heads/</code>	One file per local branch containing its commit hash
<code>.git/refs/tags/</code>	One file per tag
<code>.git/refs/remotes/</code>	Remote-tracking branch refs
<code>git reflog</code>	All HEAD movements — safety net for lost commits
<code>git reflog expire --all</code>	Expire old reflog entries (before gc)
<code>git gc</code>	Pack loose objects, prune dangling refs, compress
<code>git fsck</code>	Verify object database integrity; list dangling objects

COMMON MISTAKES

- › Running `git gc --prune=now` immediately after an accidental reset — this permanently removes the dangling commit before you can recover it via `git reflog`
- › Assuming `git branch -d` frees disk space — deleted branches leave dangling objects until the next `git gc` prune cycle
- › Treating short hashes as stable identifiers — a 7-char abbreviation can become ambiguous as the repo grows; always store full 40-char hashes in scripts

TIPS

- › Use `git cat-file -p` and `git ls-tree` when debugging unexpected behavior — seeing the raw objects demystifies what merge, rebase, and reset actually change
- › Run `git fsck` if a repo seems corrupt after a crash — it identifies missing or broken objects before `gc` removes them
- › For large repos with binary assets, consider **Git LFS** (Large File Storage) — it replaces blobs with small pointer files and stores the actual data on a separate server

SUMMARY

- › Four object types: `blob` · `tree` · `commit` · `tag`
- › `git cat-file -p` — inspect any object by hash
- › Refs in `.git/refs/` — human-readable pointers to hashes
- › `git fsck` — verify integrity; find dangling objects
- › `git gc` — pack loose objects, reclaim space, prune unreachable

ADVANCED GIT FEATURES

git worktree lets you check out multiple branches simultaneously in separate directories, all sharing one `.git/` — ideal for reviewing a PR while keeping your current work intact. **git submodule** embeds one Git repo inside another at a fixed commit, tracking external dependencies as part of the parent repo. **sparse-checkout** checks out only a subset of a large monorepo's file tree, dramatically reducing disk use. **git filter-repo** (a third-party tool that replaced `filter-branch`) rewrites history at scale — removing secrets, splitting repos, or extracting subdirectories.

KEY FACTS

- › Worktrees share `.git/` — no duplicate object store
- › Each worktree must be on a different branch
- › Submodule state is committed as a special `gitlink` entry in the parent tree
- › `git submodule update --init --recursive` — required after cloning a repo with submodules
- › `filter-repo` must be installed separately: `pip install git-filter-repo`
- › `git bundle` packages a repo for offline transfer

NOTES

- `worktree add` — creates a directory linked to the same `.git/`; you cannot check out the same branch in two worktrees simultaneously
- `sparse-checkout --cone` — cone mode uses directory prefixes, much faster than the older pattern-matching mode for large repos
- `filter-repo` — always work on a fresh clone with `--mirror`; the tool refuses to run on repos with uncommitted changes
- `--replace-text` — takes a file of `old==>new` lines; use regex with `regex:pattern==>replacement` syntax
- `git bundle` — the bundle file is a valid Git remote; you can fetch individual refs from it without a full clone
- After `filter-repo`, all collaborators must re-clone — the rewritten history is incompatible with existing local clones

WORKTREE, SPARSE-CHECKOUT & FILTER-REPO

```
# — Review a PR without losing your current work —————
git fetch origin
git worktree add -b review/pr-42 ../pr-42-review origin/feature/pr-42
cd ../pr-42-review    # full checkout on the PR branch
# review, run tests, leave notes
cd ../my-main-project # back to your original work, untouched
git worktree remove ../pr-42-review

# — Work with a monorepo: only check out one package —————
git clone --no-checkout git@github.com:org/monorepo.git
cd monorepo
git sparse-checkout init --cone
git sparse-checkout set packages/ui packages/shared
git checkout main    # only packages/ui + packages/shared downloaded

# — Remove an accidentally committed secret from all history —
# BACKUP FIRST: git clone --mirror origin backup-repo
pip install git-filter-repo
# Create replacements file:
echo "my_secret_token==>REDACTED" > secrets.txt
git filter-repo --replace-text secrets.txt
# Force push all refs (coordinate with team first):
git push origin --force --all
git push origin --force --tags

# — Bundle a repo for offline transfer —————
git bundle create /media/usb/project.bundle --all
# On another machine:
git clone /media/usb/project.bundle project
cd project
git remote set-url origin git@github.com:org/project.git
```

WORKTREE & SUBMODULE COMMANDS

COMMAND	DESCRIPTION
<code>git worktree add ../hotfix fix/crash</code>	Check out <code>fix/crash</code> branch in a sibling directory
<code>git worktree add -b review ../pr123 origin/pr123</code>	New branch from remote in a new worktree
<code>git worktree list</code>	List all linked worktrees
<code>git worktree remove ../hotfix</code>	Remove a linked worktree
<code>git submodule add <url> path/</code>	Embed a repo as a submodule at <code>path/</code>
<code>git submodule update --init --recursive</code>	Initialize and clone all submodules after cloning parent
<code>git submodule foreach git pull</code>	Run a command in every submodule
<code>git submodule deinit path/</code>	Remove submodule from working tree (keeps <code>.gitmodules</code>)

SPARSE CHECKOUT, BUNDLE & FILTER-REPO

COMMAND	DESCRIPTION
<code>git sparse-checkout init --cone</code>	Enable cone-mode sparse checkout (faster pattern matching)
<code>git sparse-checkout set packages/ui</code>	Check out only the <code>packages/ui</code> subtree
<code>git sparse-checkout disable</code>	Return to full checkout
<code>git bundle create repo.bundle --all</code>	Pack entire repo into a single transportable file
<code>git clone repo.bundle my-clone</code>	Clone from a bundle file (works offline)
<code>git filter-repo --path src/ --invert-paths</code>	Remove the <code>src/</code> directory from all history
<code>git filter-repo --path src/</code>	Keep only <code>src/</code> in history (extract a subdirectory)
<code>git filter-repo --replace-text secrets.txt</code>	Replace strings matching patterns across all blobs

COMMON MISTAKES

- › Running `git filter-repo` without first backing up the repo — history rewriting is irreversible; always work from a `--mirror` clone
- › Cloning a submodule repo without `--recursive` — submodule directories exist but are empty; run `git submodule update --init --recursive` to populate them
- › Trying to check out the same branch in two worktrees — Git prevents this; create a new branch in the second worktree with `-b`

TIPS

- › Use `git worktree` instead of stashing when switching contexts — full working tree isolation means no risk of applying the wrong stash later
- › If a repo has a `.gitmodules` file, add `git submodule update --init --recursive` to your project's setup script so new contributors don't get caught by empty submodule dirs
- › Before using `filter-repo` to remove secrets, rotate all exposed credentials immediately — assume the secret is already compromised if it was ever pushed

SUMMARY

- › `git worktree add` — multiple branches checked out simultaneously
- › `git submodule update --init --recursive` — populate submodules after clone
- › `sparse-checkout --cone` — partial monorepo checkout by directory
- › `git filter-repo` — rewrite history to remove files or secrets
- › `git bundle` — portable offline repo transport format